

run_cread.py (Q&A Documentation)

Q: What does this run script control? **A:** It controls the full experiment loop: args, data loading, model instantiation, training with a specific loss/optimizer, and evaluation with shared metrics.

Q: Why is the model not responsible for training/evaluation? **A:** Keeping training logic in the script makes it easy to compare models under the same protocol and avoids mixing I/O/metrics with differentiable code.

Q: What does this file export (public API)? **A:** The public API is the set of classes/functions other modules import (sometimes re-exported by package initializers). The key top-level definitions detected here are:

```
def get_args(...) # function
def get_loaders(...) # function
def discretize_time_label(...) # function
def restore_time_label(...) # function
def get_ord_criterion(...) # function
def get_split_nodes(...) # function
def cread_grid_search(...) # function
def mae_rescale_to_second(...) # function
def test(...) # function
```

Q: What are the main dependencies (imports) and why do they matter? **A:** Imports show whether this file is model code (torch), data code (pandas), or evaluation/analysis (sklearn). They also determine runtime requirements.

```
import os
import copy
import torch
import random
import numpy as np
import argparse
from dataloader import KUAIRECDataLoader
from model import Cread
from utils import eval_mae, eval_xauc, eval_kl
```

```
def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuairec')
    parser.add_argument('--dataset_path', default='./dataset/')
    parser.add_argument('--device', default='cuda:0')
    parser.add_argument('--bsz', type=int, default=2048)
```

```

parser.add_argument('--log_interval', type=int, default=10)

parser.add_argument('--epoch', type=int, default=10)
parser.add_argument('--restore_w', type=float, default=1.0)
parser.add_argument('--ord_w', type=float, default=0.00002)
parser.add_argument('--lr', type=float, default=0.1)
parser.add_argument('--weight_decay', type=float, default=1e-6)
parser.add_argument('--bkt_num', type=float, default=50)
parser.add_argument('--seed', type=int, default=42)

args = parser.parse_args()
return args

def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuaiREC':
        dataloaders = KUAIRECDataloader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unkown dataset name: {}'.format(name))
    return dataloaders

def discretize_time_label(playtime, split_nodes):
    playtime = playtime.reshape([-1, 1, 1])
    split_nodes = split_nodes.reshape([1, 1, -1])
    cmp_tensor = playtime > split_nodes
    binary_labels = torch.where(cmp_tensor, torch.ones_like(cmp_tensor), torch.zeros_like(cmp_tensor))
    return torch.squeeze(binary_labels)

def restore_time_label(preds, split_nodes):
    append_split_nodes = torch.concat((torch.tensor([0]).to(torch.float32).to(split_nodes.device), split_nodes), dim=0)
    left_split_nodes, right_split_nodes = append_split_nodes[:-1], append_split_nodes[1:]
    bkt_size_list = right_split_nodes - left_split_nodes
    return torch.sum(preds * bkt_size_list.view([1, -1]), dim=1) # [bsz]

```

Q: What is get_args and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for get_args? **A:** Excerpt from the file:

```

def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuaiREC')
    parser.add_argument('--dataset_path', default='./dataset/')
    parser.add_argument('--device', default='cuda:0')
    parser.add_argument('--bsz', type=int, default=2048)
    parser.add_argument('--log_interval', type=int, default=10)

    parser.add_argument('--epoch', type=int, default=10)
    parser.add_argument('--restore_w', type=float, default=1.0)
    parser.add_argument('--ord_w', type=float, default=0.00002)
    parser.add_argument('--lr', type=float, default=0.1)
    parser.add_argument('--weight_decay', type=float, default=1e-6)
    parser.add_argument('--bkt_num', type=float, default=50)
    parser.add_argument('--seed', type=int, default=42)

```

```
args = parser.parse_args()
return args
```

Q: What is `get_loaders` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_loaders`? **A:** Excerpt from the file:

```
def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuaiREC':
        dataloaders = KUAIRECDataLoader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unkown dataset name: {}'.format(name))
    return dataloaders
```

Q: What is `discretize_time_label` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `discretize_time_label`? **A:** Excerpt from the file:

```
def discretize_time_label(playtime, split_nodes):
    playtime = playtime.reshape([-1, 1, 1])
    split_nodes = split_nodes.reshape([1, 1, -1])
    cmp_tensor = playtime > split_nodes
    binary_labels = torch.where(cmp_tensor, torch.ones_like(cmp_tensor), torch.zeros_like(cmp_tensor)) # [bsz, 1, M]
    return torch.squeeze(binary_labels)
```

Q: What is `restore_time_label` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `restore_time_label`? **A:** Excerpt from the file:

```
def restore_time_label(preds, split_nodes):
    append_split_nodes = torch.concat((torch.tensor([0]).to(torch.float32).to(split_nodes.device), split_nodes))
    left_split_nodes, right_split_nodes = append_split_nodes[:-1], append_split_nodes[1:]
    bkt_size_list = right_split_nodes - left_split_nodes
    return torch.sum(preds * bkt_size_list.view([1, -1]), dim=1) # [bsz]
```

Q: What is `get_ord_criterion` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_ord_criterion`? **A:** Excerpt from the file:

```
def get_ord_criterion(preds):
    left_preds, right_preds = preds[:, :-1], preds[:, 1:]
    return torch.sum(torch.clamp(right_preds - left_preds, min=0.0))
```

Q: What is `get_split_nodes` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_split_nodes`? **A:** Excerpt from the file:

```
def get_split_nodes(all_labels, M, alpha):
    split_nodes = []
    cdf_list = []
    for m in range(1, M+1):
        z = m / M
        gamma = (1 - np.exp(-alpha*z)) / (1 - np.exp(-alpha))
        split_nodes.append(torch.quantile(all_labels, gamma))
        cdf_list.append(gamma)
    return torch.tensor(split_nodes), torch.tensor(cdf_list)
```

Q: What is `cread_grid_search` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `cread_grid_search`? **A:** Excerpt from the file:

```
def cread_grid_search(dataloader_train, M):
    all_labels = []
    for (_, label) in dataloader_train:
        all_labels.append(label)
    all_labels = torch.concat(all_labels).to(torch.float32)
    alpha_search_space = list(np.arange(0.001, 5.0, 0.1))
    beta_search_space = [50]
    best_loss, best_alpha, best_beta, best_split = None, None, None, None
    print("Strat Cread Split Nodes Search....1")
    for alpha in alpha_search_space:
        for beta in beta_search_space:
            split_nodes, cdf_list = get_split_nodes(all_labels, M, alpha)
            split_nodes_left, split_nodes_right = torch.cat([torch.tensor([0]), split_nodes[:-1]]), split_nodes
            cdf_list_left, cdf_list_right = torch.cat([torch.tensor([0]), cdf_list[:-1]]), cdf_list
            A_w = torch.sum(torch.pow(cdf_list_right - cdf_list_left, 2)) * torch.sum(torch.pow(split_nodes_right - split_nodes_left, 2))
            A_b = torch.sum(torch.pow(cdf_list_right - cdf_list_left, 2)) * torch.sum(torch.pow(split_nodes_right - split_nodes_left, 2))
            A_loss = A_w + beta * A_b
            print("Searching | alpha={:.7f}, beta={:.7f}: A_loss={:.7f}, A_w={:.7f}, A_b={:.7f} ".format(alpha, beta, A_loss, A_w, A_b))
            if (best_loss == None) or (best_loss > A_loss):
                best_loss, best_alpha, best_beta, best_split = A_loss, alpha, beta, split_nodes
    print("Cread Search Complete! Best Loss is {:.7f}, Best Alpha is {:.7f}, Best Beta is {:.7f}.".format(best_loss, best_alpha, best_beta))
    return best_split
```

Q: What is `mae_rescale_to_second` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `mae_rescale_to_second`? **A:** Excerpt from the file:

```
def mae_rescale_to_second(dataset, mae):
    if dataset == 'kuaiirec':
        return mae * 999639 / 1000
    elif dataset == 'wechat':
        return mae * 20840
    elif dataset == 'cikm16':
```

```

        return (mae *(6000-31) + 31) / 1000
    else:
        raise ValueError('unkown dataset name: {}'.format(dataset))

```

Q: What is test and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for test? **A:** Excerpt from the file:

```

def test(args, model, dataloaders):
    model.eval()
    labels, scores, predicts = list(), list(), list()
    with torch.no_grad():
        for _, (features, label) in enumerate(dataloaders['test']):
            preds = model(features)
            y = restore_time_label(preds, split_nodes)
            labels.extend(label.tolist())
            scores.extend(y.tolist())
    labels, scores = np.array(labels), np.array(scores)
    mae, xauc, kl = eval_mae(labels, scores), eval_xauc(labels, scores), eval_kl(labels, scores)
    mae = mae_rescale_to_second(args.dataset_name, mae)
    print("test result | MAE: {:.7f} | XAUC: {:.7f} | KL: {:.7f}".format(mae, xauc, kl))

```

Q: Example: end-to-end data flow involving this file **A:** Core pipeline shape (schematic):

```

python run_cread.py --dataset_name kuaiREC --dataset_path ./dataset --device cuda:0
# Internally: load dataloaders → build model → train → evaluate

```

Q: Full source code of the Python file **A:** The complete file is included verbatim for reference.

```

import os
import copy
import torch
import random
import numpy as np
import argparse
from dataloader import KUAIRECDataloader
from model import Cread
from utils import eval_mae, eval_xauc, eval_kl

def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuaiREC')
    parser.add_argument('--dataset_path', default='./dataset/')
    parser.add_argument('--device', default='cuda:0')
    parser.add_argument('--bsz', type=int, default=2048)
    parser.add_argument('--log_interval', type=int, default=10)

    parser.add_argument('--epoch', type=int, default=10)
    parser.add_argument('--restore_w', type=float, default=1.0)
    parser.add_argument('--ord_w', type=float, default=0.00002)
    parser.add_argument('--lr', type=float, default=0.1)
    parser.add_argument('--weight_decay', type=float, default=1e-6)
    parser.add_argument('--bkt_num', type=float, default=50)
    parser.add_argument('--seed', type=int, default=42)

    args = parser.parse_args()
    return args

def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuaiREC':

```

```

        dataloaders = KUAIRECDataLoader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unkown dataset name: {}'.format(name))
    return dataloaders

def discretize_time_label(playtime, split_nodes):
    playtime = playtime.reshape([-1, 1, 1])
    split_nodes = split_nodes.reshape([1, 1, -1])
    cmp_tensor = playtime > split_nodes
    binary_labels = torch.where(cmp_tensor, torch.ones_like(cmp_tensor), torch.zeros_like(cmp_tensor)) # [bsz, 1, M]
    return torch.squeeze(binary_labels)

def restore_time_label(preds, split_nodes):
    append_split_nodes = torch.concat((torch.tensor([0]).to(torch.float32).to(split_nodes.device), split_nodes))
    left_split_nodes, right_split_nodes = append_split_nodes[:-1], append_split_nodes[1:]
    bkt_size_list = right_split_nodes - left_split_nodes
    return torch.sum(preds * bkt_size_list.view([1, -1]), dim=1) # [bsz]

def get_ord_criterion(preds):
    left_preds, right_preds = preds[:, :-1], preds[:, 1:]
    return torch.sum(torch.clamp(right_preds - left_preds, min=0.0))

def get_split_nodes(all_labels, M, alpha):
    split_nodes = []
    cdf_list = []
    for m in range(1, M+1):
        z = m / M
        gamma = (1 - np.exp(-alpha*z)) / (1 - np.exp(-alpha))
        split_nodes.append(torch.quantile(all_labels, gamma))
        cdf_list.append(gamma)
    return torch.tensor(split_nodes), torch.tensor(cdf_list)

def cread_grid_search(dataloader_train, M):
    all_labels = []
    for (_, label) in dataloader_train:
        all_labels.append(label)
    all_labels = torch.concat(all_labels).to(torch.float32)
    alpha_search_space = list(np.arange(0.001, 5.0, 0.1))
    beta_search_space = [50]
    best_loss, best_alpha, best_beta, best_split = None, None, None, None
    print("Strat Cread Split Nodes Search....1")
    for alpha in alpha_search_space:
        for beta in beta_search_space:
            split_nodes, cdf_list = get_split_nodes(all_labels, M, alpha)
            split_nodes_left, split_nodes_right = torch.cat([torch.tensor([0]), split_nodes[:-1]]), split_nodes
            cdf_list_left, cdf_list_right = torch.cat([torch.tensor([0]), cdf_list[:-1]]), cdf_list
            A_w = torch.sum(torch.pow(cdf_list_right - cdf_list_left, 2)) * torch.sum(torch.pow(split_nodes_right - split_nodes_left, 2))
            A_b = torch.sum(torch.pow(cdf_list_right - cdf_list_left, 2)) * torch.sum(torch.pow(split_nodes_right - split_nodes_left, 2))
            A_loss = A_w + beta * A_b
            print("Searching | alpha={:.7f}, beta={:.7f}: A_loss={:.7f}, A_w={:.7f}, A_b={:.7f} ".format(alpha, beta, A_loss, A_w, A_b))
            if (best_loss == None) or (best_loss > A_loss):
                best_loss, best_alpha, best_beta, best_split = A_loss, alpha, beta, split_nodes
    print("Cread Search Complete! Best Loss is {:.7f}, Best Alpha is {:.7f}, Best Beta is {:.7f}.".format(best_loss, best_alpha, best_beta))
    return best_split

def mae_rescale_to_second(dataset, mae):
    if dataset == 'kuaiREC':
        return mae * 999639 / 1000
    elif dataset == 'wechat':
        return mae * 20840
    elif dataset == 'cikm16':
        return (mae * (6000-31) + 31) / 1000
    else:
        raise ValueError('unkown dataset name: {}'.format(dataset))

def test(args, model, dataloaders):
    model.eval()
    labels, scores, predicts = list(), list(), list()

```

```

with torch.no_grad():
    for _, (features, label) in enumerate(dataloaders['test']):
        preds = model(features)
        y = restore_time_label(preds, split_nodes)
        labels.extend(label.tolist())
        scores.extend(y.tolist())
labels, scores = np.array(labels), np.array(scores)
mae, xauc, kl = eval_mae(labels, scores), eval_xauc(labels, scores), eval_kl(labels, scores)
mae = mae_rescale_to_second(args.dataset_name, mae)
print("test result | MAE: {:.7f} | XAUC: {:.7f} | KL: {:.7f}".format(mae, xauc, kl))

if __name__ == '__main__':
    args = get_args()
    if args.seed > -1:
        np.random.seed(args.seed)
        torch.manual_seed(args.seed)
        torch.cuda.manual_seed(args.seed)
    res = {}
    torch.cuda.empty_cache()

    device = torch.device(args.device)

    # construct DataLoader
    dataloaders = get_loaders(args.dataset_name, args.dataset_path, device, args.bsz)

    # discretization strategy
    split_nodes = cread_grid_search(dataloaders['train'], args.bkt_num).to(device)
    print("split node list: ", split_nodes)
    M = split_nodes.shape[0]

    # construct model
    model = Cread(dataloaders.description, embed_dim=16, share_mlp_dims=(512, 256, 128), output_mlp_dims=(64, 32), head_num=M,
    model = model.to(device)

    # train
    dataloader_train = dataloaders['train']
    model.train()
    bce_criterion = torch.nn.BCELoss()
    huber_criterion = torch.nn.HuberLoss()
    optimizer = torch.optim.Adam(params=model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
    for epoch_i in range(1, args.epoch + 1):
        model.train()
        epoch_loss, epoch_loss_bce, epoch_loss_restore, epoch_loss_ord = 0.0, 0.0, 0.0, 0.0
        total_loss, total_loss_bce, total_loss_restore, total_loss_ord = 0.0, 0.0, 0.0, 0.0
        total_iters = len(dataloader_train)
        for i, (features, label) in enumerate(dataloader_train):
            preds = model(features) # [bsz, M]
            binary_labels = discretize_time_label(label, split_nodes)
            restore_pred = restore_time_label(preds, split_nodes)
            loss_bce = bce_criterion(preds, binary_labels.float())
            loss_restore = huber_criterion(restore_pred, label.float())
            loss_ord = get_ord_criterion(preds)
            loss = loss_bce + args.restore_w * loss_restore + args.ord_w * loss_ord
            model.zero_grad()
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item(); epoch_loss_bce += loss_bce.item(); epoch_loss_restore += args.restore_w * loss_restore.it
            total_loss += loss.item(); total_loss_bce += loss_bce.item(); total_loss_restore += args.restore_w * loss_restore.it
            if (i + 1) % 10 == 0:
                print("    Iter {}/{} loss: {:.7f}, loss_bce: {:.7f}, loss_restore: {:.7f}, loss_ord: {:.7f} ".format(i + 1, t
                    total_loss = 0
        print("Epoch {}/{} average Loss: {:.7f}, loss_bce: {:.7f}, loss_restore: {:.7f}, loss_ord: {:.7f}".format(epoch_i, args.
    test(args, model, dataloaders)

```